

Introduction to regression with gradient descent

Latreche Sara

Contents

1	Introduction to Regression and Optimization	3
2	Linear regression	3
3	Linear Regression: Learning from Data	4
4	Linear Regression Equation in Machine Learning	6
5	Models with Multiple Features	7
6	The Final Linear Regression Model	8
7	Linear Regression: Loss	9
7.1	Understanding Loss as Distance	10
7.2	Types of Loss in Linear Regression	10
7.3	Choosing Between MAE and MSE	12
7.4	Linear Regression: Gradient Descent	13
8	Algorithmic Diagnosis and Visual Intuition	18
8.1	The Gradient Descent Path on Contour Plots	19
8.2	Derivative Behavior and Quiver Plots	19
8.2.1	The Quiver Plot as a Vector Field	20
8.3	Comparison: Machine Learning vs. Statistics	20

9	Multiple Linear Regression	20
9.1	Model and Cost Function	21
9.2	Gradient Descent with Multiple Features	22
9.3	Feature Mapping and Polynomial Regression	25
10	Model Fit and Generalization	26
10.1	Analysis of Fitting Scenarios	26
10.2	Regularization	27
11	Conclusion	28

1 Introduction to Regression and Optimization

Regression is a fundamental technique used to find and quantify the mathematical relationship between features and labels . In both the fields of statistics and machine learning, the primary objective of regression is to predict a continuous numerical value—such as a car’s fuel efficiency or the market price of a home—based on specific input data .While traditional statistics often focuses on inference and understanding the underlying relationship through fixed formulas, machine learning emphasizes predictive performance and the model’s ability to generalize to new, unseen data . To achieve this, machine learning employs an optimization engine known as **Gradient Descent**. This is an iterative algorithm that ”learns” the optimal parameters—specifically the weights and the bias—by following the slope of the cost function until it converges at the global minimum.

While this course begins with Linear Regression, there are various advanced algorithms available to solve complex regression problems, including **Support Vector Regression (SVR)**, **Decision Trees**, and powerful ensemble methods such as **XGBoost**, **Gradient Boosting Machines (GBM)**, and **AdaBoost** [1]. To ensure these models generalize effectively and avoid overfitting, regularization techniques like **Ridge (L2)** and **Lasso (L1)** are employed. These methods add a penalty to the cost function to discourage the weights from becoming excessively large, with Lasso specifically capable of performing feature selection by pushing less important weights to zero [1, 2]. Regression remains the industry standard for foundational training due to its efficiency in diverse real-world applications, such as forecasting patient recovery times in healthcare or predicting economic trends [1]. By starting with gradient descent, we establish a deep understanding of how model parameters are updated based on the magnitude and direction of the gradient, ensuring the path toward the minimum is steady and monotonic [2].

2 Linear regression

Definition

Linear regression models the relationship between an input variable x and an output variable y by fitting a straight line:

$$\hat{y} = wx + b$$

Our goal is to find parameters w (weight) and b (bias) that minimize the difference between predicted and actual values.

3 Linear Regression: Learning from Data

Linear regression is a fundamental statistical technique used to understand the relationship between two variables. In the context of **machine learning (ML)**, it helps us model the relationship between **features** (inputs) and a **label** (output).

Real-World Example: Predicting Weight from Height

Example

Suppose you want to predict a person's **weight** (in kilograms) based on their **height** (in centimeters). You collect the following data:

Height (cm)	Weight (kg)
150	50
160	55
170	65
180	72
190	80

When you plot this data, you'll likely observe a **positive trend**—taller people generally weigh more.

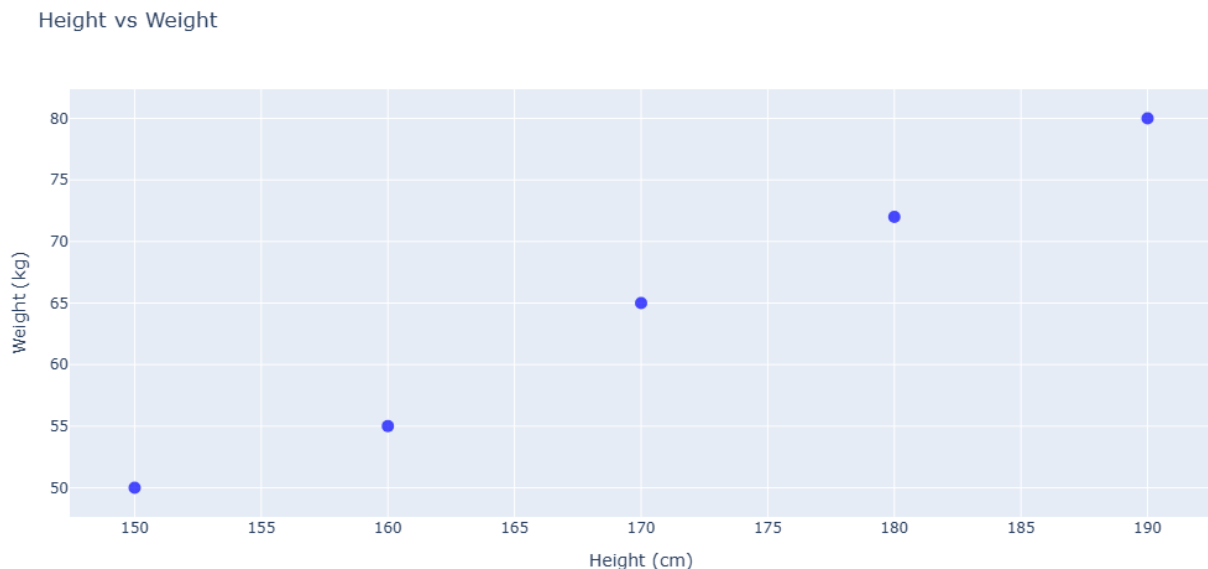


Figure 1: Scatter Plot of Height vs Weight (Without Best Fit Line)

We could create our own model by drawing a best fit line through the points:

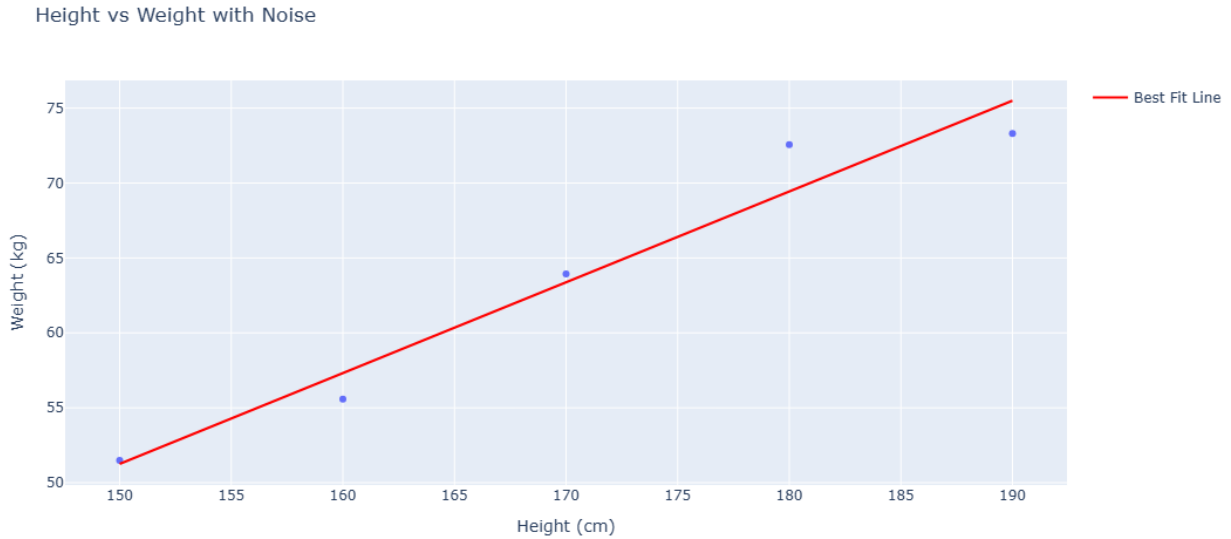


Figure 2: Height vs Weight with Best Fit Line

Linear regression helps us fit a straight line that best represents this relationship. The model takes the form of a simple equation:

$$\hat{y} = w \cdot x + b$$

Where:

- $\hat{y}$ is the predicted **weight**,
- x is the input **height**,
- w is the **slope** of the line (also called the weight of the model),
- b is the **bias** or **intercept**—where the line crosses the y-axis.

Example Model

Let's say we trained a linear model and found:

$$w = 0.6 \quad \text{and} \quad b = -40$$

Then the model becomes:

$$\hat{y} = 0.6 \cdot x - 40$$

So, for a person who is 170 cm tall:

$$\hat{y} = 0.6 \cdot 170 - 40 = 62 \text{ kg}$$

Linear Model Prediction at Height = 170 cm

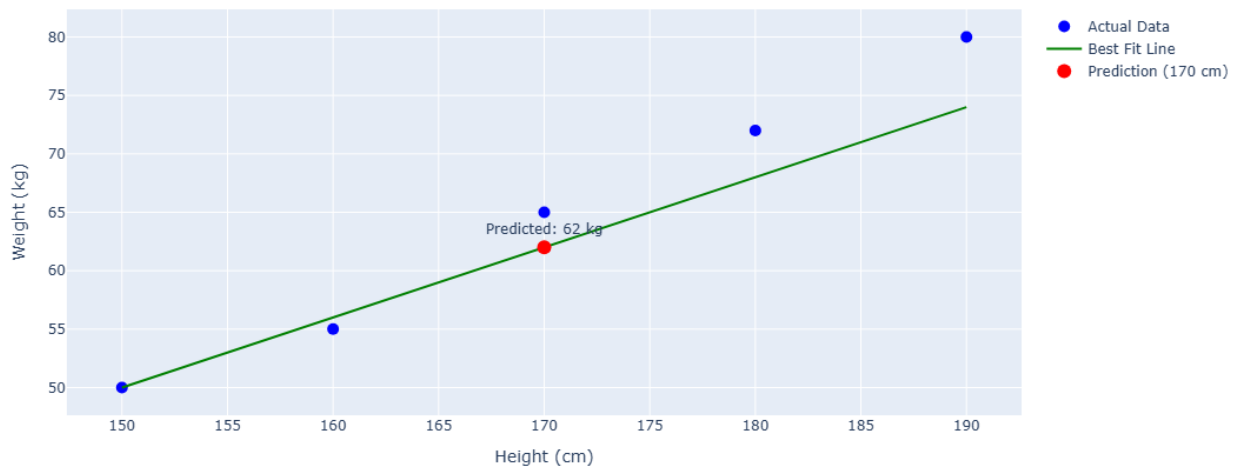


Figure 3: Prediction for Height = 170 cm Using the Learned Line

4 Linear Regression Equation in Machine Learning

Definition

In **algebraic terms**, the linear regression model is defined by the equation of a line:

$$y = w \cdot x + b$$

Note

In **Machine Learning (ML)**, we emphasize that the output is an estimate by using the "hat" notation ($\hat{y}$):

$$\hat{y} = w \cdot x + b$$

Variable Breakdown:

- $\hat{y}$ (**Predicted Label**): The output of the model (e.g., predicted *miles per gallon*).
- x (**Input Feature**): The independent variable (e.g., *car weight*).
- w (**Weight**): Equivalent to the *slope*. It reflects the strength and direction of the relationship between the feature and output, learned during training.
- b (**Bias**): Equivalent to the *y-intercept*. It allows the model to shift the prediction line to better fit the data.

5 Models with Multiple Features

So far, we've used only one feature—like a person's height—to predict weight. However, real-world predictions often depend on a combination of factors.

Definition

Multiple Linear Regression

If a model uses n features $(x_1, x_2, \dots, x_n)$, the prediction equation generalizes to:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

Variable Definitions:

- $\hat{y}$: The **predicted output** (e.g., weight in kg).
- b : The **bias** term.
- w_j : The **weight** assigned to the j -th feature, representing its relative importance.
- x_j : The j -th **input feature**.

Practical Application: Predicting Weight

Example

Suppose we predict weight based on three features: **Height** (x_1), **Age** (x_2), and **Shoe Size** (x_3).

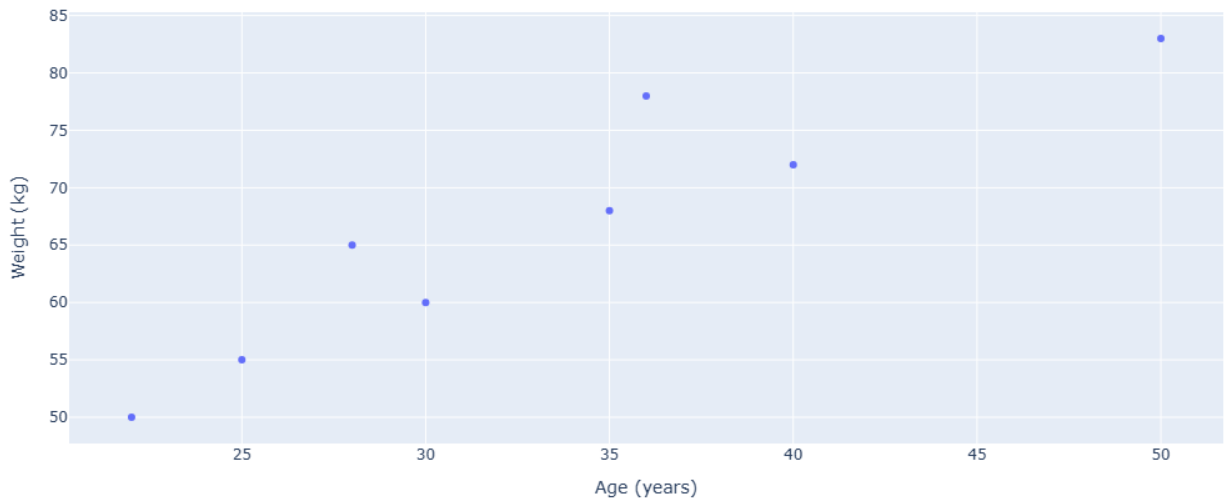
Height (x_1)	Age (x_2)	Shoe Size (x_3)	Weight (y)
150	22	38	50
160	25	39	55
170	28	42	65
180	40	44	72
190	50	46	83

In this case, the model would learn three different weights (w_1, w_2, w_3) to determine how much each characteristic influences the final weight.

Visualizing Feature Relationships

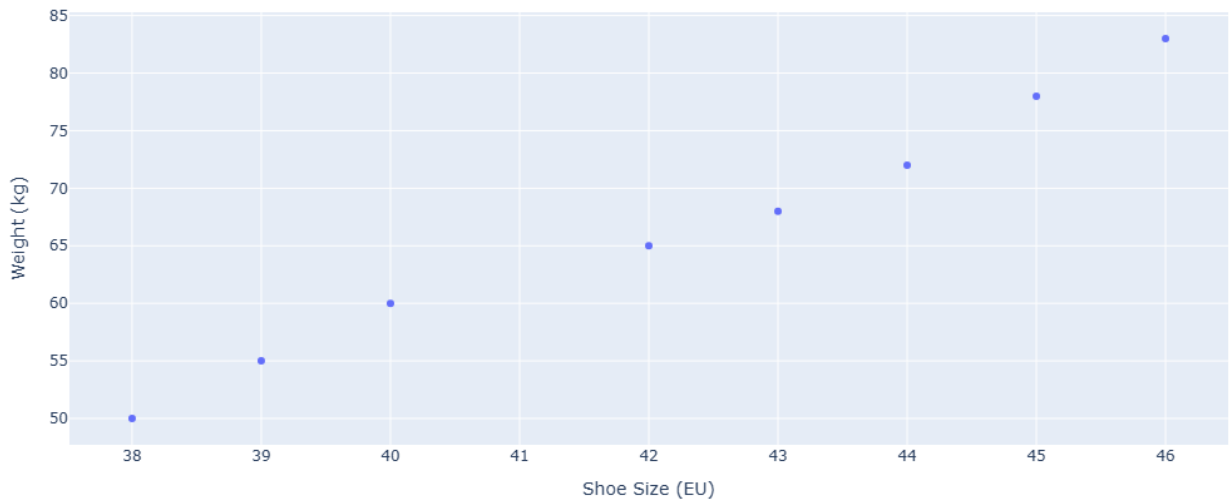
By plotting these additional features against weight, we observe they exhibit linear trends:

Figure 2: Age vs Weight



•

Figure 3: Shoe Size vs Weight



•

6 The Final Linear Regression Model

After the training process (using Gradient Descent), the model "learned" the following specific parameters that minimize our error:

- **Weights:** $w_1 = 0.393$, $w_2 = 0.143$, $w_3 = 0.882$
- **Bias:** $b = -58.492$

Definition

The Prediction Equation

Substituting our learned parameters into the general form, we get:

$$\hat{y} = 0.393(x_1) + 0.143(x_2) + 0.882(x_3) - 58.492$$

Step-by-Step Example Prediction

Example

Let's predict the weight of a person with: **Height** = 170, **Age** = 28, and **Shoe Size** = 42.

1. Substitution:

$$\hat{y} = (0.393 \cdot 170) + (0.143 \cdot 28) + (0.882 \cdot 42) - 58.492$$

2. Intermediate Calculation:

$$\hat{y} = 66.81 + 4.004 + 37.044 - 58.492$$

3. Final Result:

$$\hat{y} = 49.366 \text{ kg}$$

Note

This demonstrates the power of **Multiple Linear Regression**. By learning separate weights (w_i) for each feature, the model captures complex patterns that a single-feature model would miss. Each weight tells us the "influence" of that specific feature on the final weight.

7 Linear Regression: Loss

In machine learning, **loss** is a key concept that helps in evaluating the performance of a model. Specifically for linear regression, the goal is to minimize the difference between the predicted values and the actual target values, i.e., the labels.

Loss in Linear Regression

In linear regression, we aim to fit a line to the data points so that the predictions of the model are as close as possible to the actual data points. The loss function quantifies the error (the difference) between the actual data points and the model's predictions.

Figure 2: Age vs Weight

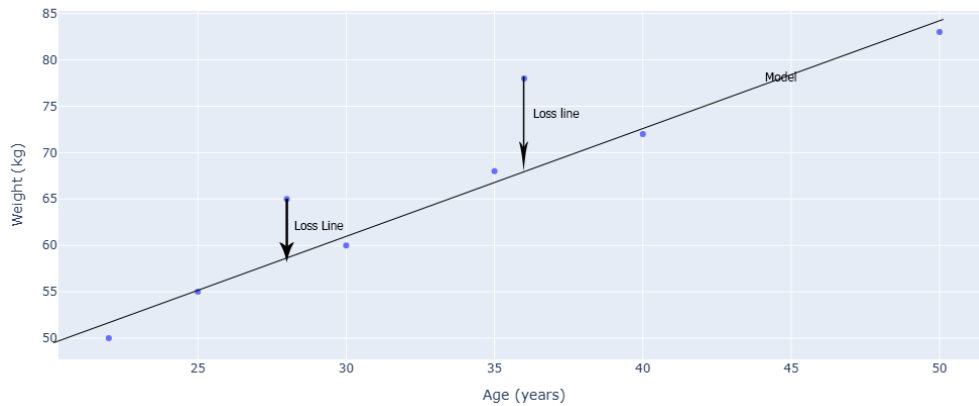


Figure 4. Loss is measured from the actual value to the predicted value.

Note

In some textbooks, the loss function is also called the **cost function** and is often denoted by the symbol J . This function represents the overall error of the model and serves as the objective we want to minimize during training.

7.1 Understanding Loss as Distance

In machine learning and statistics, **loss** quantifies how far off a model's prediction is from the actual value. Rather than focusing on whether the model overestimated or underestimated, loss measures the magnitude of the error — the distance between prediction and reality — regardless of direction.

For instance, if a model predicts a value of 2 when the true value is 5, what matters isn't that the error is -3 , but that the prediction is 3 units away from the actual value. To emphasize this, most loss functions eliminate the sign of the error.

The most common techniques to achieve this are:

- Taking the absolute value of the difference between prediction and actual value.
- Squaring the difference, which also penalizes larger errors more heavily.

Both approaches ensure that the model's learning focuses on reducing the size of the error, not its direction.

7.2 Types of Loss in Linear Regression

In linear regression, there are four main types of loss, which are outlined in the following table:

Loss Type	Definition	Equation
L1 Loss	Sum of absolute differences	$L1 = \sum y_i - \hat{y}_i $
MAE	Average of L1 losses	$MAE = \frac{1}{n} \sum y_i - \hat{y}_i $
L2 Loss	Sum of squared differences	$L2 = \sum (y_i - \hat{y}_i)^2$
MSE	Average of L2 losses	$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$

Table 1: Common loss functions used in linear regression

The functional difference between **L1 loss** and **L2 loss** (or between **MAE** and **MSE**) is **squaring**:

- When the difference between the prediction and label is **large**, squaring makes the loss **even larger**.
- When the difference is **small** (less than 1), squaring makes the loss **even smaller**.

Note

Tip: When processing multiple examples, it is recommended to **average the losses** across all examples—whether using MAE or MSE.

Example

Numerical Example: Calculating Loss Functions

Using the example dataset from Section 1.2.1, consider the following data point:

- Height: 170 cm, Age: 28 years, Shoe Size: 42 EU
- Actual Weight (y): 65 kg

Step 1: Calculate the predicted weight ($\hat{y}$)

$$\hat{y} = 0.393(170) + 0.143(28) + 0.882(42) - 58.492 = \mathbf{49.366 \text{ kg}}$$

Step 2: Calculate the error

$$\text{Error} = y - \hat{y} = 65 - 49.366 = 15.634$$

Step 3: Calculate the L1 loss (absolute error)

$$L1 = |15.634| = \mathbf{15.634}$$

Step 4: Calculate the L2 loss (squared error)

$$L2 = (15.634)^2 = \mathbf{244.42}$$

Interpretation: The L1 loss shows the magnitude of the error directly (15.634 kg), while the L2 loss penalizes larger errors more strongly (244.42).

7.3 Choosing Between MAE and MSE

When selecting a loss function for regression tasks, it is important to understand that **sensitivity to outliers** is a key—but not the only—factor influencing the choice between **Mean Absolute Error (MAE)** and **Mean Squared Error (MSE)**.

Outliers:

- **MSE** penalizes large errors more heavily by squaring the differences. This makes it very sensitive to outliers and encourages the model to fit these extreme values more closely.
- **MAE** treats all errors linearly, making it more robust to outliers.

Mathematical Properties:

- **MSE** is **differentiable everywhere**, making it easier and faster to optimize using gradient-based methods like stochastic gradient descent.
- **MAE** is **not differentiable at zero**, which can make optimization less stable and slower.

Interpretability:

- **MAE** is easier to interpret—it directly tells you the average error magnitude (in the same units as the label).
- **MSE** can be harder to interpret due to squaring the errors, but it emphasizes larger mistakes.

Data Distribution:

- If the errors in your data are **normally distributed**, **MSE** is statistically optimal.
- For data with **heavy tails or skewed error distributions**, **MAE** may yield better generalization.

Note

Use Case Summary

Criterion	Prefer MSE	Prefer MAE
Handle Outliers	High penalty	Robust
Optimization	Smooth gradients	Non-differentiable at 0
Interpretability	Less intuitive	Direct average error
Error Distribution	Normal	Heavy-tailed/Skewed

In practice, it is often helpful to **experiment with both loss functions** during model development and use validation metrics to determine which performs best on your specific dataset.

7.4 Linear Regression: Gradient Descent

Gradient Descent Explained with Height vs. Weight Gradient descent is an optimization technique that adjusts model parameters (weight and bias) to minimize the **loss function**—commonly the Mean Squared Error (MSE) in regression problems. In this case, we train a model to predict **weight** (y) based on **height** (x):

Definition

The Linear Hypothesis

$$\hat{y} = w \cdot x + b$$

- w : Weight (slope)
- b : Bias (intercept)

Step-by-Step Process

1. **Initialize parameters:** Set $w = 0$, $b = 0$ (or small random values).
2. **Compute predictions:** $\hat{y}_i = w \cdot x_i + b$.
3. **Calculate the loss:** Use Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

4. **Compute gradients:** Calculate the partial derivatives:

$$\frac{\partial \text{MSE}}{\partial w} = \frac{2}{n} \sum_{i=1}^n x_i (\hat{y}_i - y_i)$$
$$\frac{\partial \text{MSE}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

5. **Update parameters:** Move opposite the gradient:

$$w := w - \alpha \cdot \frac{\partial \text{MSE}}{\partial w} \quad , \quad b := b - \alpha \cdot \frac{\partial \text{MSE}}{\partial b}$$

Note

The Learning Rate (α)

- A **small** α leads to slow but stable convergence.
- A **large** α can cause the model to overshoot and fail to converge.

Repeat steps 2–5 until the loss converges (stops changing significantly). In real-world scenarios, training continues until the model reaches a **stable state**. At this point, further updates no longer lead to meaningful reductions in loss.

Key Point: Convergence is reached when the loss plateaus. The algorithm may still make tiny adjustments, causing the loss to hover slightly around a minimum value; this indicates the model has settled.

Algorithm 1 Gradient Descent for Linear Regression

```
1: Initialize:  $w \leftarrow 0, b \leftarrow 0$ 
2: Input: Learning rate  $\alpha$ , Iterations  $T$ , Dataset size  $n$ 
3: for  $t = 1$  to  $T$  do
4:   Step 1: Compute Predictions
5:    $\hat{y}_i = w \cdot x_i + b$  for all  $i \in \{1, \dots, n\}$ 
6:   Step 2: Calculate Gradients
7:    $dw = \frac{2}{n} \sum_{i=1}^n x_i(\hat{y}_i - y_i)$ 
8:    $db = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$ 
9:   Step 3: Update Parameters
10:   $w \leftarrow w - \alpha \cdot dw$ 
11:   $b \leftarrow b - \alpha \cdot db$ 
12: end for
13: return Final parameters  $w, b$ 
```

Numerical Application of Gradient Descent

To see Gradient Descent in action, let's perform one full iteration manually. This will help us understand how the parameters w and b shift in response to the error.

Example

Setup and Initialization

Using our Height vs. Weight dataset ($n = 5$):

Height (x)	150	160	170	180	190
Weight (y)	50	55	65	72	80

Hyperparameters: $w = 0, \quad b = 0, \quad \alpha = 0.0001$

Iteration 0: The First Step

1. **Initial Predictions ($\hat{y}$):** Since $w, b = 0$, all predictions are $\hat{y}_i = 0$.
2. **Error Calculation ($\hat{y}_i - y_i$):** The initial errors are simply $-50, -55, -65, -72, -80$.
3. **Compute Gradients:**

$$\frac{\partial \text{MSE}}{\partial w} = \frac{2}{5} \sum x_i(\hat{y}_i - y_i) = \frac{2}{5}(-55, 510) = -22, 204$$

$$\frac{\partial \text{MSE}}{\partial b} = \frac{2}{5} \sum (\hat{y}_i - y_i) = \frac{2}{5}(-322) = -128.8$$

4. Update Parameters:

$$w := 0 - (0.0001 \times -22,204) = \mathbf{2.2204}$$

$$b := 0 - (0.0001 \times -128.8) = \mathbf{0.01288}$$

Analyzing the Result: Overshooting

With our new parameters, let's see how the model performs:

$$\hat{y}_i = 2.2204(x_i) + 0.01288$$

Height (x_i)	Actual (y_i)	Prediction ($\hat{y}_i$)	Error
150	50	333.06	+283.06
160	55	355.27	+300.27
170	65	377.48	+312.48

Note

Warning: Overshoot Detected!

Notice that the errors increased significantly (from -50 to $+283$). This indicates that the ****learning rate (α) is too large****. The step we took was so big that we jumped over the "valley" of the loss function and landed even higher on the other side.

Summary of Parameter Evolution:

Iteration	w	b	Grad w	Grad b
0	0	0	-22,204	-128.8
1	2.2204	0.01288	<i>Recalculating...</i>	<i>Recalculating...</i>

Visualization of Gradient Descent

The visualization below illustrates how the prediction line gradually improves as the model updates w and b over time, effectively reducing the loss and fitting the data more closely.

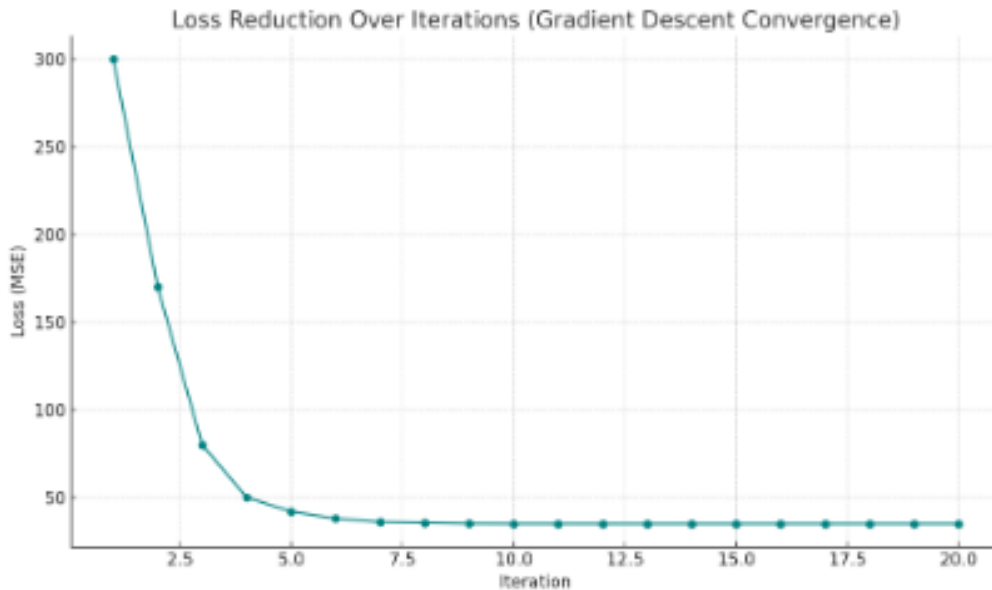


Figure 4: Visualizing parameter updates: The regression line converging to the data points.

Note

The Importance of the Learning Rate (α)

- **Too high ($\uparrow$):** The model may overshoot the minimum and diverge (as seen in our numerical example).
- **Too low ($\downarrow$):** The convergence becomes slow, making training computationally expensive and inefficient.

Convergence and Convex Functions

In linear regression, the **Mean Squared Error (MSE)** forms what is known as a **convex surface** when plotted against the parameters w and b .

Definition

What is Convexity?

A convex function is shaped like a "bowl." This unique mathematical property ensures:

- There is only **one global minimum** (the absolute lowest point).
- There are **no local minima** to get stuck in.
- Gradient descent is guaranteed to find the optimal solution if the learning rate is tuned correctly.

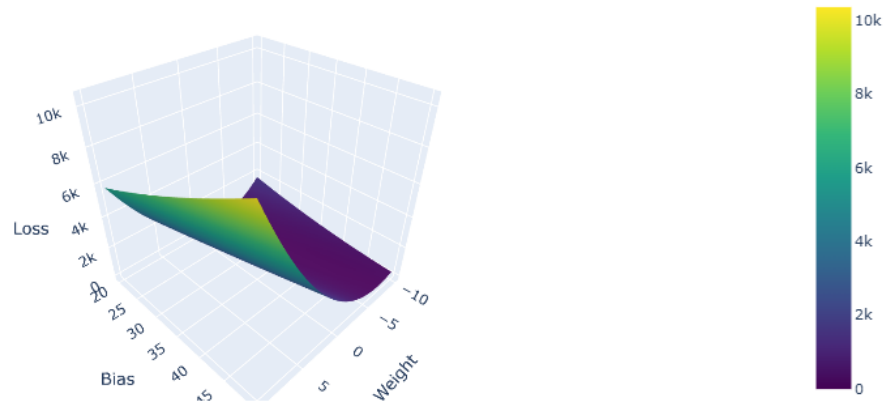


Figure 5: Convex loss surface: gradient descent "rolling" toward the global minimum.

This process is analogous to a ball rolling down a hill toward the bottom of a bowl:

- **Optimal α :** The ball rolls smoothly and settles at the bottom.
- **Large α :** The ball gains too much momentum, overshooting the bottom and oscillating wildly.
- **Small α :** The ball moves at a snail's pace, taking far too long to reach the minimum.

Note

Generalization

While we focused on Height vs. Weight, Gradient Descent is the engine behind:

- **Deep Learning:** Training neural networks with millions of parameters.
- **Multiple Features:** Handling hundreds of inputs simultaneously.
- **Diverse Loss Functions:** Optimizing for classification, ranking, and more.

8 Algorithmic Diagnosis and Visual Intuition

In Machine Learning, diagnosing the training process is critical to ensure that the parameter vector $\mathbf{w}$ and bias b are converging efficiently. This differs from classical statistics by focusing on the behavior of the gradient updates within the optimization landscape.

8.1 The Gradient Descent Path on Contour Plots

The first diagnostic tool is the **Contour Plot**, which provides a top-down view of the cost surface $J(\mathbf{w}, b)$.

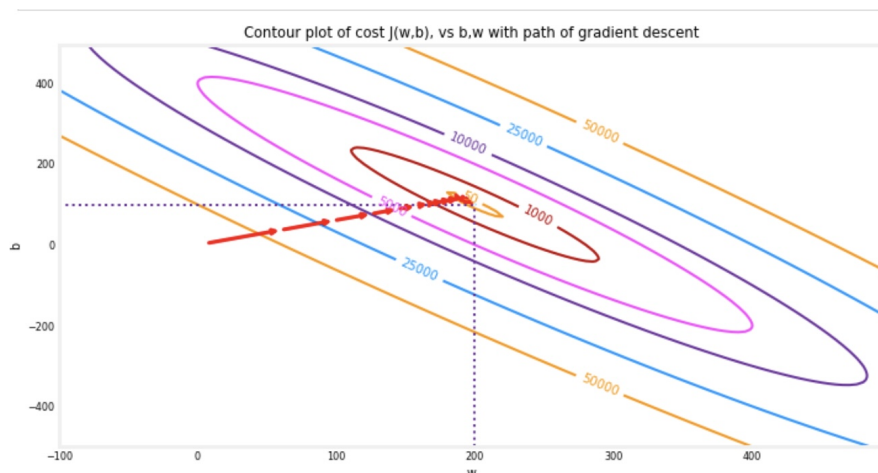


Figure 6: Contour plot of cost $J(\mathbf{w}, b)$ showing the monotonic path of gradient descent toward the global minimum.

Note

Adaptive Step Size Observation

Notice in Figure 6 that the initial steps are much larger than the steps near the goal. This occurs naturally because the gradient magnitude decreases as the slope flattens, causing the updates to shrink as the algorithm reaches the minimum.

8.2 Derivative Behavior and Quiver Plots

To understand why the algorithm moves in a specific direction, we analyze the relationship between the cost curve and the gradient field.

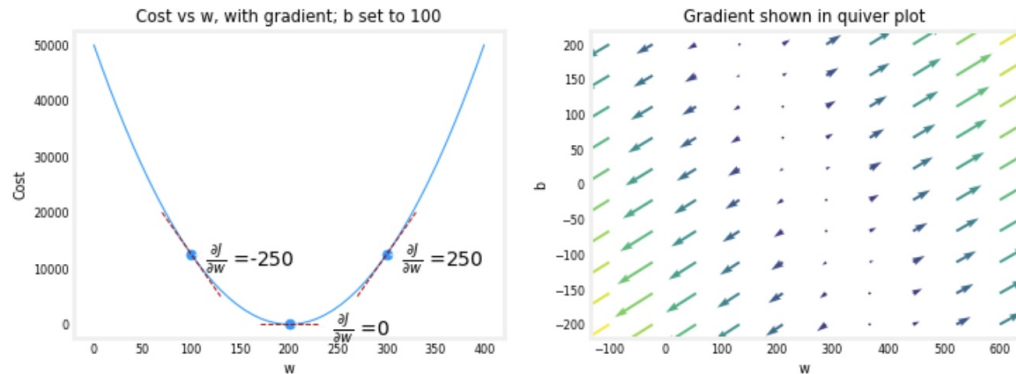


Figure 7: Left: Cost vs. w illustrating derivative signs. Right: Quiver plot visualizing the gradient vector field.

8.2.1 The Quiver Plot as a Vector Field

The quiver plot provides a means of viewing the gradient of both parameters simultaneously:

- **Arrow Direction:** Arrows point *away* from the minimum (steepest ascent). By subtracting the gradient, we move in the opposite direction.
- **Arrow Size:** The magnitude of the gradient is reflected in the size of the arrows; steeper regions produce larger updates.

8.3 Comparison: Machine Learning vs. Statistics

In Machine Learning, we use these visuals to diagnose **Optimization Health**:

Symptom	Visual Observation	ML Diagnosis
Oscillation	Dots jump back and forth	Learning rate α is too high.
Stagnation	Arrows are tiny, but cost is high	Model may need Feature Scaling .
Overfitting	Train cost ≈ 0 , Val cost is high	High variance; needs regularization.

9 Multiple Linear Regression

We now extend our algorithm to handle a feature matrix $\mathbf{X}$ of shape (m, n) . Our parameters transition from scalars to a weight vector $\mathbf{w}$ and a scalar bias b .

Definition

Assumptions of Multiple Linear Regression

- **Linearity:** The target y is a linear combination of features in $\mathbf{X}$.
- **No Multicollinearity:** Features should not be highly correlated. High correlation creates "flat" regions in $J(\mathbf{w}, b)$, leading to slow convergence.
- **Scale of Features:** Gradient Descent performs significantly better when features are on a similar scale (e.g., via **Z-score Normalization**).

Note

Matrix Notation

Instead of writing $y = w_1x_1 + w_2x_2 + \dots$, we use the dot product:

$$\hat{y} = \mathbf{w}^T \mathbf{X} + b$$

This notation is not only cleaner but is computationally optimized for modern hardware (GPUs).

9.1 Model and Cost Function

In the multi-feature setting, we represent our parameters and data as vectors. The prediction for a single training example $\mathbf{x}^{(i)}$ is calculated efficiently using the dot product:

Definition

The Multivariate Hypothesis

$$f_{\mathbf{w},b}(\mathbf{x}^{(i)}) = \mathbf{w} \cdot \mathbf{x}^{(i)} + b$$

Where:

- $\mathbf{w}$ is the weight vector $[w_0, w_1, \dots, w_{n-1}]$
- $\mathbf{x}^{(i)}$ is the feature vector for the i -th example
- b is the scalar bias

To measure how well our model is performing across all m training examples, we define the **Cost Function** $J(\mathbf{w}, b)$. This represents the average of the squared differences between the predicted values and the actual targets:

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=0}^{m-1} (f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

Note

Why the 2 in the denominator?

You may notice we use $\frac{1}{2m}$ instead of $\frac{1}{m}$. This is a common mathematical convenience: when we take the derivative (gradient) of the squared term $(\dots)^2$, the 2 from the exponent cancels out with the 2 in the denominator, leaving us with a much cleaner gradient update rule.

9.2 Gradient Descent with Multiple Features

To minimize the cost function, we use Gradient Descent. This requires calculating the partial derivatives (gradients) for each parameter w_j and the bias b .

Algorithm 2 Gradient Descent for Multiple Linear Regression

- 1: **Initialize:** $\mathbf{w} \leftarrow [0, \dots, 0]$, $b \leftarrow 0$
- 2: **Input:** Learning rate α , Iterations T , Examples m , Features n
- 3: **for** $t = 1$ **to** T **do**
- 4: **Step 1: Compute Gradients**
- 5: For each feature $j \in \{0, \dots, n - 1\}$:

$$\frac{\partial J(\mathbf{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{\mathbf{w}, b}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- 6: For the bias b :

$$\frac{\partial J(\mathbf{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=0}^{m-1} (f_{\mathbf{w}, b}(\mathbf{x}^{(i)}) - y^{(i)})$$

- 7: **Step 2: Simultaneous Update**
 - 8: $w_j \leftarrow w_j - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial w_j}$ for all j
 - 9: $b \leftarrow b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$
 - 10: **end for**
 - 11: **return** $\mathbf{w}, b$
-

Note

Important: Simultaneous Updates

When implementing this in code, ensure that you calculate the gradients for ****all**** w_j and b using the old values of the parameters before updating any of them. If you update w_0 and then use that new value to calculate the gradient for w_1 , you are no longer performing standard Gradient Descent!

Note

Parameters vs. Hyperparameters

- **Parameters** (w, b): Values the model learns automatically from the data.
- **Hyperparameters** (α , batch size): Settings you define *before* training to guide the learning process.

Common Hyperparameters

- **Learning rate** (α): Controls the step size during gradient descent.
- **Batch size**: Number of examples used in one iteration to compute the gradient.
- **Epochs**: One full pass through the entire training dataset.

Learning Rate Analysis

The learning rate determines if your model successfully **converges**.

- **Too low**: Slow, inefficient convergence.
- **Too high**: Erratic behavior, "bouncing," or complete divergence.

Update Example: If the gradient is 2.5 and $\alpha = 0.01$:

$$\text{Change} = 0.01 \times 2.5 = 0.025$$

$$w_{\text{new}} = w_{\text{old}} - 0.025$$

Batch Size and Training Dynamics

Batch size defines how many training examples are processed before updating parameters.

Machine Learning Training Concepts

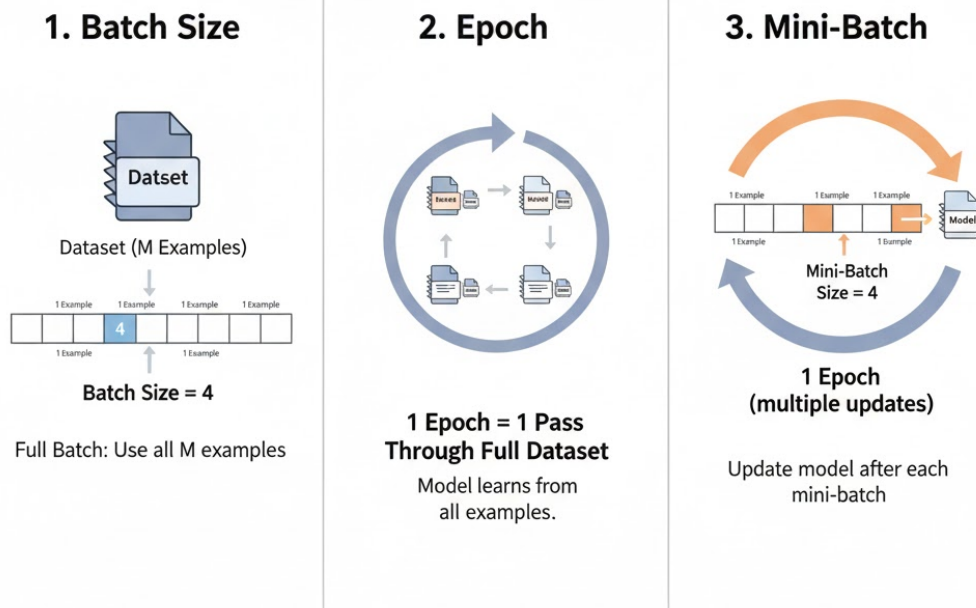


Figure 8: Visualizing Batch, Epoch, and Mini-Batch dynamics.

Strategy	Examples per Update	Key Characteristic
Batch GD	Entire Dataset (m)	Stable but very slow for big data.
Mini-Batch GD	Small group (e.g., 32, 64)	Best balance of speed and stability.
Stochastic (SGD)	1 example	Very fast but very noisy path.

Mini-Batch SGD: Mini-batching is the industry standard. It utilizes modern CPU/GPU vectorization while providing enough noise to help the model avoid getting stuck in "flat" regions of the cost surface.

Noise and Its Role in Training: Noise may seem undesirable, but a controlled amount of noise—often introduced by Mini-Batch SGD—helps the model avoid local minima and improves generalization.

Note

Benefits of Controlled Noise:

- **Generalization:** Prevents the model from "memorizing" specific data points (overfitting).
- **Exploration:** Helps the algorithm "jump" out of small, shallow local minima.
- **Global Search:** Encourages the path toward a more robust global solution.

Epochs in Training

An **epoch** represents a full pass through the entire training dataset. For example, with 1,000 training samples and a batch size of 100, one epoch involves 10 iterations.

- Multiple epochs are typically required for the model to "see" the data enough times to learn.
- **Too few epochs:** The model underfits (hasn't learned enough).
- **Too many epochs:** The model may overfit or waste computation time.

9.3 Feature Mapping and Polynomial Regression

Real-world data is rarely a perfect straight line. When patterns are curved, we use **feature mapping** to expand our feature set.

Definition

Polynomial Feature Mapping

We transform the input x into a polynomial feature vector:

$$\phi(x) = [1, x, x^2, x^3, \dots, x^M]$$

The regression model then becomes:

$$y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M$$

Key Insight: Even though the graph is curved, this is still a "Linear Model" because it is **linear in the weights (w)**. This means we can still use Gradient Descent to solve it!

10 Model Fit and Generalization

A critical part of the machine learning process is ensuring that the model generalizes well to new data. We analyze this through the concepts of **Bias** (underfitting) and **Variance** (overfitting).

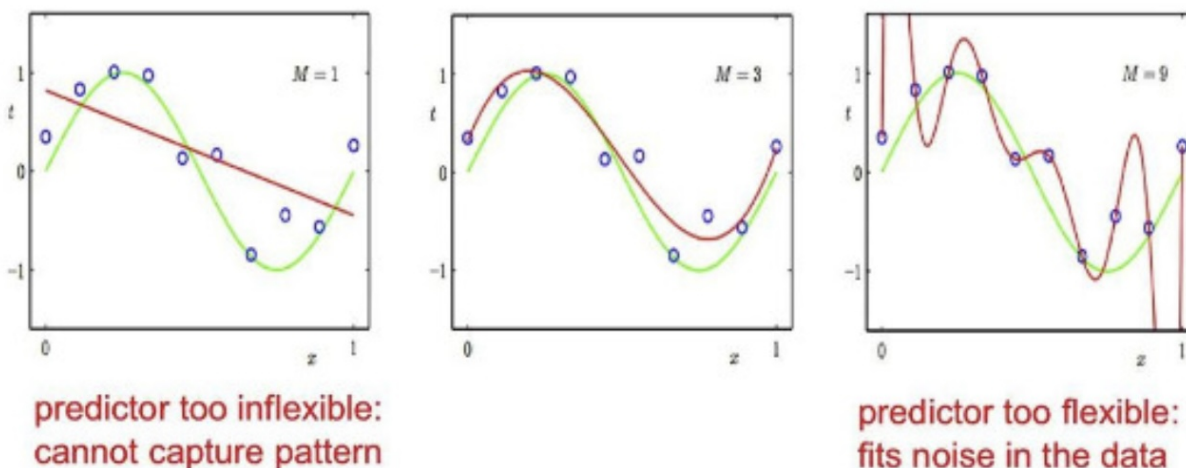


Figure 9: Comparison of model complexity: Underfitting (left), Ideal Fit (center), and Overfitting (right).

Condition	Cause	Solution
Underfitting	Model is too simple (High Bias)	Add features or increase M .
Ideal Fit	Captures the underlying trend	Keep as is.
Overfitting	Model is too complex (High Variance)	Use Regularization or more data.

10.1 Analysis of Fitting Scenarios

As illustrated in Figure 9, we identify three distinct behaviors based on model flexibility:

- **Underfitting (High Bias):** The predictor is too inflexible (e.g., a straight line for curved data). The model is too simple for the complexity of the task.
- **Ideal Fit:** The model captures the general trend without being distracted by individual noise points.
- **Overfitting (High Variance):** The predictor is too flexible ($M = 9$) and fits the noise rather than the pattern. While training error may be near zero, it fails on unseen data.

10.2 Regularization

To prevent overfitting when using many features or high-degree polynomials, we apply **regularization**. This adds a penalty to the loss function to discourage the model from assigning excessively large weights, keeping the model "simpler."

Example

ML Diagnostic Rationale

In Machine Learning, we diagnose the model by looking at the gap between training and validation performance:

1. **To Fix Underfitting:** Increase model complexity (add features or polynomial terms).
2. **To Fix Overfitting:** Decrease complexity, gather more data, or apply **Regularization**.

Types of Regularization

The hyperparameter λ (lambda) controls the "strength" of the penalty.

Definition

Mathematical Formulations

- **L2 Regularization (Ridge):** Penalizes the square of the weights.

$$\text{Loss}_{\text{ridge}} = \text{MSE} + \lambda \sum_{j=1}^n w_j^2$$

- **L1 Regularization (Lasso):** Penalizes the absolute value of the weights.

$$\text{Loss}_{\text{lasso}} = \text{MSE} + \lambda \sum_{j=1}^n |w_j|$$

Note

The Role of λ

If λ is set to 0, you are performing standard Linear Regression. If λ is too large, the penalty becomes so heavy that all weights approach zero, leading back to **underfitting**. Choosing the right λ is a critical part of hyperparameter tuning.

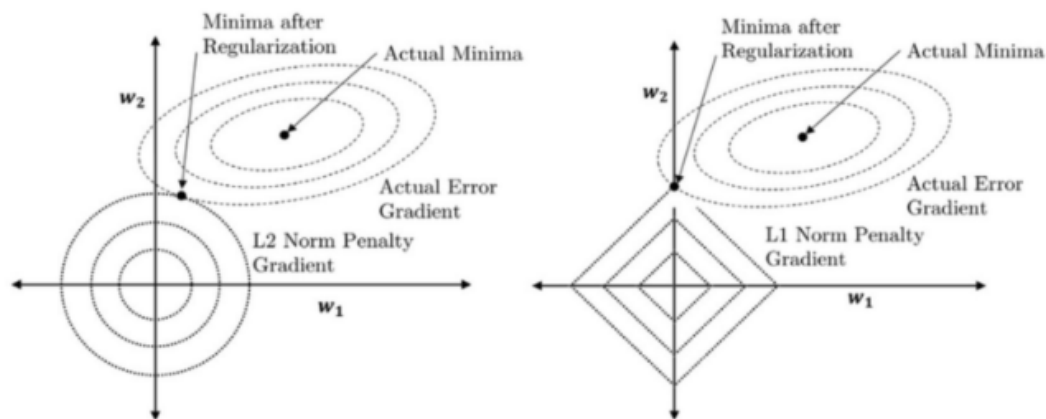


Figure 10: **Geometric Comparison of L1 (Lasso) and L2 (Ridge) Regularization.**

Description: This visualization explains why L1 and L2 behave differently:

- **L1 (Diamond):** The sharp corners of the L1 "diamond" constraint often intersect with the cost contours exactly on an axis. This forces some weights (w_j) to become exactly **zero**, performing automatic feature selection.
- **L2 (Circle):** The smooth, circular constraint of L2 typically intersects the contours at non-zero points. This shrinks all weights to be **very small** but rarely zero, keeping all features in the model.

Key Idea: Regularization balances the trade-off between fitting the training data and keeping the model simple enough to generalize to the real world.

11 Conclusion

Throughout this course, we have seen how **Linear Regression** serves as the bedrock of predictive modeling. By moving from simple algebraic lines to multiple features and polynomial transformations, we have developed a framework capable of modeling complex real-world phenomena.

Note

The Power of Gradient Descent

Gradient descent is the engine that makes this possible. By iteratively refining parameters (w, b) to minimize a loss surface, it enables models to be trained efficiently on datasets far too large for classical closed-form solutions.

Looking Ahead: This optimization process is not unique to linear models; it serves as the fundamental foundation for nearly all modern machine learning methods. Whether you are building a recommendation system or a deep neural network, the principles of gradients, learning rates, and convergence remain exactly the same.

Example

Final Takeaway for Latreche Sara's Students:

- **Start Simple:** Always begin with a basic linear model and MSE loss.
- **Monitor Health:** Use contour and loss plots to diagnose convergence.
- **Generalize:** Use regularization to ensure your model works on data it has never seen before.

Congratulations on completing the Introduction to Regression with Gradient Descent!

References

- [1] Google Developers. *Régression linéaire — Machine Learning Crash Course*. Google for Developers. Available at: <https://developers.google.com/machine-learning/crash-course/linear-regression?hl=fr>
- [2] Andrew Ng. *Machine Learning Specialization*. Coursera (DeepLearning.AI & Stanford University). Available at: <https://www.coursera.org/specializations/machine-learning-introduction>